Naming conventions, Ruff, and stage 1 clean-up

From the SCRUM-2 follow-up pass on src/bronze/data_ingestion.py: fixing the duplicate zero-row log, renaming the ingestion functions, and setting up Ruff so formatting stops taking up review rounds.

01 Keep the try block narrow -- don't catch your own raise

The zero-row check used to sit inside the same try as the read. The raise ValueError was caught by the generic except Exception right below it, which logged a second error ('Failed to read') with a traceback, then re-raised. One empty file produced two error logs, and the second one was wrong: the read had succeeded.

Rule: a try should wrap only the lines that can raise the errors its except is meant to handle. Technical failures (missing file, bad header, S3 down) and your own business rules (zero rows) are different things -- keep the business rule after the try/except.

```
try:
    df = spark.read....csv(...)
    df = df.cache()
    row_count = df.count()
except Exception:
    logger.exception("Failed to read %s", file_name)
    raise

if row_count == 0:
    logger.error("There are no rows in the file %s", file_name)
    raise ValueError(f"There are no rows in the file {file_name}")
```

Log once, with the right message. Removing the logger.error entirely was the opposite mistake: nothing in app.py catches the error, so it would only appear as an uncaught traceback on stderr, outside the logging setup. Long-term pattern: the entry point (main) catches, logs and exits non-zero.

02 cache() is lazy -- the first action fills it

cache() does not load anything. It only marks the DataFrame to be kept the next time an action (count, write, collect) runs. So the order matters:

```
df = df.cache()        # mark
row_count = df.count() # action: reads CSV once, fills cache
write(df)              # action: served from cache
```

Moving cache() after count() made the CSV get parsed twice (once by count, once by write) plus build a cache nobody read -- worse than no cache at all. Leaving an empty DataFrame cached on the error path costs nothing; the job stops anyway.

Habit: when moving a line near a Spark action, ask which action will run it and how many times the source gets read.

03 Naming functions -- what was wrong and why

Before	After	Why
loader	land_raw_file_in_bronze	Noun -> verb phrase. Says what moves where. 'Land' is the ELT word.
bronze_layer_loader	(rejected)	Still a noun; 'layer' is noise.
read_data / write_data	read_raw_file / write_raw_file	'data' says nothing; 'raw' says which layer.

Before	After	Why
read_csv_files	(rejected)	Plural is wrong: it reads ONE file. Format-only name says nothing new.
test_ingest_dimensions.py	test_data_ingestion.py	Covered reviews too (not a dimension). Test file mirrors the module.

Checks to run on any function name:

- Is it a verb phrase? Functions do things: land_, read_, write_, build_, dedup_.
- Read the call site out loud: land_raw_file_in_bronze(spark=..., file_name=...) should sound like a description of the step.
- Does every word carry meaning? Drop noise words: data, info, layer, manager, helper.
- Is singular/plural accurate? One file -> file. A table -> table.
- Will it still be clear once silver and gold exist next to it?
- Repeating the package name (bronze) is fine only if it makes the call site clearer.

04 Python naming conventions (PEP 8) -- reference

Kind	Convention	Example
Function / method	snake_case, verb phrase	read_raw_file, dedup_reviews
Variable / parameter	snake_case, noun	row_count, source_path
Boolean	is_ / has_ / should_ prefix	is_delivered, has_review
Class	PascalCase, noun	BronzeReader, CustomerDimension
Constant (module level)	UPPER_SNAKE_CASE	BRONZE_PATH, SCHEMAS
Module / file	short snake_case	data_ingestion.py
Internal (not public)	leading underscore	_build_path
Test function	test_<behaviour>_<condition>	test_zero_row_file_raises

Methods follow the same rules as functions. The class name already gives context, so don't repeat it: CustomerDimension.load(), not CustomerDimension.load_customer_dimension().

05 Type hints -- documentation the tools can read

Python does not check type hints when code runs. They tell the editor and type checkers (mypy) what a parameter is, so autocomplete works on spark.read and wrong arguments get flagged.

```
from pyspark.sql import DataFrame, SparkSession

def read_raw_file(spark: SparkSession, source_path: str, ...) -> DataFrame:
```

Annotate every function that takes the parameter -- especially the public entry point, which was the one missed first time. Formatting: a space after the colon (spark: SparkSession, not spark:SparkSession).

06 Don't move code to utils until it actually repeats

Idea: move read/write to utils so other flows can use them. Pushed back, because:

- Only one caller exists today -- that's designing ahead of need.
- The functions aren't generic: CSV-only, multiline, enforceSchema, zero-row rule. Silver writes Parquet, so reuse would mean adding format=, options=, check_empty= flags -- harder to read than two small specific

functions.
- The real repeat will probably be narrower ('read a bronze table'), and its home may be next to bronze, not in a generic utils.

Rule: extract once the pattern really repeats, and decide the shape (functions vs class) with two concrete examples in front of you.

07 Formatting is a tool's job -- Ruff

Four review rounds went mostly on trailing spaces, blank lines, paren placement and import order. In production this is never reviewed by hand. Three layers:

Layer	What	Where
1. Config in repo	One formatter + rules everyone shares	pyproject.toml
2. Editor	Format + sort imports on every save	.vscode/settings.json
3. Enforcement	Block unformatted commits / PRs	pre-commit hook, CI

Ruff replaces black (layout), isort (imports) and flake8 (lint) with one tool.

```
[tool.ruff]
line-length = 100
target-version = "py310"   # Spark container runs Python 3.10

[tool.ruff.lint]
select = ["E", "W", "F", "I"]

[tool.ruff.lint.isort]
known-first-party = ["src"]
```

```
ruff format .         # fix layout
ruff check --fix .    # sort imports, fix lint
ruff format --check . # CI-style check, changes nothing
```

target-version is the LOWEST Python the code runs on (the container), not the one on your laptop -- otherwise ruff can produce syntax the container rejects.

Magic trailing comma: ruff collapses a parameter list onto one line if it fits. Put a comma after the last parameter and ruff keeps one-per-line.

What no tool fixes: the content of comments and names. That's what code review is for.

08 Habits before asking for a review

- Run pytest. A rename broke test collection (ImportError on 'loader') and no test ran.
- After a rename, grep for the old names across src/ and tests/.
- Tests must call your code, not Spark directly. The 3 existing tests call spark.read with their own options -- removing multiline from read_raw_file would not fail any of them. (Next: item 3.)
- Local tests need Java 17 -- Spark 3.4 fails to start on Java 21.
- Use git mv (or git add -A) so a renamed file is recorded as a rename.
- One commit = one coherent change, with a message that says what's in it.